Graph Theory Coding Task
Group 6



Group Members :
Lina Fatima Azzahra Badr - 5025251168
Naila Sa'ada Cahyani - 5025251258

A. Introduction

For this coding task, we are tasked with using graph theory to implement graph algorithms in order to solve three tasks:

1. Task 1 (SPPG Simulator)
2. Task 2 (DLH Simulator)
3. Task 3 (Knight's Tour Simulator)

B. System Architecture

We use python for the tasks' programs. We separate the algorithm programs (knight_tour.py and chinese_postman.py) and the data ingestion programs (engine folder). After downloading the repository, the tasks' programs can be run from the repository folder as instructed in the readme file.

C. Task 1

1. Problem

We are tasked with finding an optimal Travelling Salesman Problem (TSP) closed delivery route for an SPPG delivering MBG mealboxes to schools in Surabaya. The route must start and end at the chosen SPPG location and visit every school within a selected radius (x km) exactly once with the minimum total travel distance and time, using the actual road network.

2. Data

The map used for the road network is sourced from OpenStreetMap and modeled as a Directed MultiGraph (*DiGraph*), where edges store physical road lengths, speed limits, and travel times. School locations are also obtained from OpenStreetMap using the school tag and filtered based on the delivery radius x km from the SPPG location.

3. Algorithms

- **Road Snapping and Distance Matrix**

  Before finding the route, each stop (the SPPG depot and all schools) is snapped to the nearest road network node within a 500-meter limit. The algorithm then runs Dijkstra's shortest path algorithm from every stop to every other stop on the directed road graph to build an asymmetric distance matrix.

  From road_tour.py:

```
# Snap SPPG and schools to the nearest road network nodes
nodes = []
snap_distances = []
for stop in stops:
    node, distance = nearest_node(graph, stop.latitude, stop.longitude)
    if distance > MAX_SNAP_M:
        raise ValueError(f"{stop.name} is {distance:.0f} m from the nearest road "
                         f"(limit is {MAX_SNAP_M} m)")
    nodes.append(node)
    snap_distances.append(distance)
# Compute pairwise shortest road distances via Dijkstra on DiGraph
matrix = []
for i, source in enumerate(nodes):
    lengths = nx.single_source_dijkstra_path_length(graph, source, weight="length")
    row = []
    for j, target in enumerate(nodes):
        if target not in lengths:
            raise ValueError(f"no road path from {stops[i].name} to {stops[j].name}")
        row.append(lengths[target])
    matrix.append(row)
```

- **Held-Karp Dynamic Programming**

  To find the minimum-cost Hamiltonian cycle visiting every school and returning to the SPPG, the algorithm uses the Held-Karp dynamic programming algorithm with bitmasking. Instead of checking every permutation, it stores the cheapest

cost to visit a subset of schools ending at a specific school in a lookup table dp[(mask, last)]. Parent pointers are recorded so the exact optimal route can be reconstructed by backtracking.

From `held_karp.py`

```
# DP over bitmask subsets of visited cities
for mask in range(1, full_mask + 1):
    for last in range(others):
        if not mask & (1 << last):
            continue
        prev_mask = mask ^ (1 << last)
        if prev_mask == 0:
            continue
        best = math.inf
        best_prev = None
        for prev in range(others):
            if prev_mask & (1 << prev):
                cost = dp[(prev_mask, prev)] + matrix[prev + 1][last + 1]
                if cost < best:
                    best = cost
                    best_prev = prev
        dp[(mask, last)] = best
        parent[(mask, last)] = best_prev

# Close the loop back to city 0 (SPPG)
best_cost = math.inf
last = None
for i in range(others):
    cost = dp[(full_mask, i)] + matrix[i + 1][0]
    if cost < best_cost:
        best_cost = cost
        last = i
```

- **Route Reconstruction**
  Once Held-Karp determines the optimal visiting order of the schools, the algorithm maps the path back to the actual road network. It finds the shortest sequence of road segments between each consecutive stop, extracts the GPS coordinates along the streets, and calculates the total driving distance and travel time.
  From road_tour.py

```
total_distance = 0.0
total_travel_time = 0.0
coordinates = [(graph.nodes[nodes[0]]["y"], graph.nodes[nodes[0]]["x"])]
for a, b in zip(route, route[1:]):
    path = nx.shortest_path(graph, nodes[a], nodes[b], weight="length")
    for u, v in zip(path, path[1:]):
        # Take the shortest edge if parallel edges exist
        data = min(graph[u][v].values(), key=lambda d: d["length"])
        total_distance += data["length"]
        total_travel_time += data["travel_time"]
        points = edge_points(graph, u, v, data)
        if points[0] == coordinates[-1]:
            points = points[1:]
        coordinates.extend(points)
```

4. Complexity

- **Time:**
  Without dynamic programming, checking all possible routes (brute-force) yields a worst-case factorial time of O(N!), where N is the number of locations. But with Held-Karp, dynamic programming with bitmasking reduces this to $O(N^2 \cdot 2^N)$ time. Building the distance matrix using Dijkstra's algorithm across the road network has a time complexity of $O(N \cdot (E + V \log V))$. Because N is capped at 1
- **Space:**

$O(V + E + N \cdot 2^N)$, where $O(V + E)$ is used to store the road network graph in memory and $O(N \cdot 2^N)$ is used for the Held-Karp dynamic programming table and parent backtracking pointers.

D. Task 2

1. **Problem**

   We are tasked with finding an efficient CPP (Chinese Postman Problem) closed walk route within a DLH Dump Truck's service area. The route must traverse every single street (edge) in a directed road network (Surabaya) at least once with the minimum total travel distance.

2. **Data**

   The map used for the road network is sourced from OpenStreetMap, filtered by highway classifications (trunk, primary, secondary, tertiary). The map is modeled as a Directed MultiGraph (*DiGraph*) where edges store physical road lengths and names.

3. **Algorithms**

   - Eulerian Circuit & Degree Imbalance Check

     Before finding a route, the algorithm checks whether the directed graph is already Eulerian (balanced in-degrees and out-degrees for all nodes) and identifies unbalanced nodes.

     The algorithm iterates over every vertex in the road network graph G, computing the difference between its out-degree and in-degree. Nodes with excess outgoing edges are categorized as supply nodes, while nodes with excess incoming edges are categorized as demand

nodes. If both dictionaries are empty, the graph is already Eulerian.

From chinese_postman.py

```
out_deg = dict(G.out_degree())
in_deg = dict(G.in_degree())

supply = {}
demand = {}

for node in G.nodes():
    diff = out_deg.get(node, 0) - in_deg.get(node, 0)
    if diff > 0:
        supply[node] = diff # out > in (supply)
    elif diff < 0:
        demand[node] = -diff # in > out (demand)
```

- Min-Cost Flow Augmentation
  Using NetworkX's min_cost_flow solver, we can find the shortest paths connecting unbalanced nodes, augmenting the graph into a Eulerian graph. A transportation network flow model is constructed using a super_source and a super_sink, combined with Dijkstra's all-pairs shortest paths.

  From chinese_postman.py

```
G_aug = G.copy()

if supply and demand:
    flow_net = nx.DiGraph()
    source, sink = "super_source", "super_sink"
    flow_net.add_node(source)
    flow_net.add_node(sink)

    total_supply = sum(supply.values())
    for s, qty in supply.items():
        flow_net.add_edge(source, s, capacity=qty, weight=0)
    for d, qty in demand.items():
        flow_net.add_edge(d, sink, capacity=qty, weight=0)

    lengths = dict(nx.all_pairs_dijkstra_path_length(G, weight='length'))
    paths = dict(nx.all_pairs_dijkstra_path(G, weight='length'))

    for s in supply.keys():
        for d in demand.keys():
            if d in lengths.get(s, {}):
                flow_net.add_edge(s, d, capacity=float('inf'), weight=lengths[s][d])

    try:
        flow_dict = nx.min_cost_flow(flow_net, demand={sink: total_supply})
        for s in supply.keys():
            if s in flow_dict:
                for d, flow_val in flow_dict[s].items():
                    if d in demand.keys() and flow_val > 0:
                        path = paths[s][d]
                        for i in range(len(path) - 1):
                            u, v = path[i], path[i+1]
                            edge_data = G.get_edge_data(u, v)
                            if edge_data:
                                key = list(edge_data.keys())[0]
                                G_aug.add_edge(u, v, **edge_data[key])
```

- Hierholzer's Algorithm

  Once the graph has been augmented into an Eulerian directed graph, Hierholzer's algorithm extracts the final closed traversal circuit starting from the base of the DLH Dump Truck. NetworkX's built-in eulerian_circuit routine executes Hierholzer's algorithm in linear time relative to the number of edges, outputting the complete sequence of street segments the garbage truck must traverse.

```
    # Find Eulerian circuit
    try:
        route_edges = list(nx.eulerian_circuit(G_aug, source=start_node))
    except nx.NetworkXError:
        route_edges = [(u, v) for u, v in G_aug.edges()]

    return G_aug, route_edges
```

4. **Complexity**

The Degree Imbalance Checker has a time complexity of (O(V + E)), looping through all vertices V and edges E to compute the out-degree and in-degree. Dijkstra's All-Pairs Shortest Paths (APSP) has a time complexity of (O($V^2 logV + VE$)), it is run across the graph in order to find the shortest path between every pair of unbalanced supply and demand nodes. Min-Cost Flow Augmentation's time complexity is (O(V^3)), it has a number of nodes proportional to the number of supply and demand nodes (bounded by V) and edges bounded by (V^2)(every supply connected to every demand node). Solving min-cost flow on the network adds a polynomial term, which is bounded or dominated by Dijkstra's APSP phase on typical road networks. Hierholzer's algorithm has a time complexity of (O(E)), it traverses every edge of the augmented graph exactly once to build the final circuit runs in linear time relative to the number of edges (E).

Because Dijkstra's APSP's time complexity of (O($V^2 logV + VE$)) dominates the execution time over the simple degree checks and the linear graph traversals, the overall time complexity is expressed as (O($V^2 logV + VE$)).

E. Task 3

1. **Problem**

We are tasked with finding the sequence of moves of a knight on a chessboard where the knight visits every square exactly once.

2. **Data**

   The chessboard is modeled as a 2D grid coordinate system (column x row) where each square acts as a graph node connected to up to 8 valid knight move destinations.

   From knight_tour.py

```
_MOVES = (
    (1, 2),
    (2, 1),
    (2, -1),
    (1, -2),
    (-1, -2),
    (-2, -1),
    (-2, 1),
    (-1, 2),
)
```

3. **Algorithms**

   - Move Ordering using Warnsdorff's Heuristic

     Warnsdorff's rule dictates that the knight should always move to the adjacent square that has the fewest onward valid moves, greedily avoiding premature dead-ends.

     From knight_tour.py

```
def candidates(col: int, row: int) -> list[tuple[int, int]]:
    available = []
    for col_delta, row_delta in _MOVES:
        next_col = col + col_delta
        next_row = row + row_delta
        if (
            0 <= next_col < cols
            and 0 <= next_row < rows
            and not visited[next_col][next_row]
        ):
            # Warnsdorff counts each candidate's available onward moves.
            onward = sum(
                0 <= next_col + move_col < cols
                and 0 <= next_row + move_row < rows
                and not visited[next_col + move_col][next_row + move_row]
                for move_col, move_row in _MOVES
            )
            available.append((onward, next_col, next_row))
    available.sort()
    return [(next_col, next_row) for _, next_col, next_row in available]
```

- DFS Search & Backtracking Core
  The recursive search function explores candidate moves sequentially. If a dead-end is reached, it backtracks by unmarking the visited state and popping from the path stack.

  From knight_tour.py

```
def search(col: int, row: int) -> bool:
    visited[col][row] = True
    path.append((col, row))
    # The base case means every board square has been visited.
    if len(path) == cols * rows:
        return True

    for next_col, next_row in candidates(col, row):
        # Each recursive call explores one candidate branch.
        if search(next_col, next_row):
            return True

    path.pop()
    # Unmarking lets another branch reuse this square during backtracking.
    visited[col][row] = False
    return False
```

4. Complexity

- Time Complexity:
  Without heuristic guidance, exploring all potential paths yields a worst-case exponential time of $O(8^N)$, where N

= cols x rows is the number of squares. But with Warnsdorff's, the heuristic prunes the search tree so effectively that the algorithm runs in near-linear O(N) time, solving standard chessboards in milliseconds without hitting deep recursive dead-ends.

- Space Complexity:
  O(N), the memory footprint is proportional to the total number of squares N due to the 2D visited matrix lookup table, the coordinate recursion call stack, and the final path coordinate list.